

Datatypes :

use : To represent data in memory we can use datatypes.

PM on datatype:

- 1) used to represent type of variable.
- 2) datatype decides the memory size of the variable.
- 3) datatype decides the range of value which are stored in variable

JDK :

- 1) it is used to develop, modify java application.
- 2) it is used to compile java application.
- 3) it is used to execute java application.

JRE :

- 1) it is used to execute java application.

compiler :

- 1) it is used to check syntax of java programme.
- 2) converts source code (java code) into byte code.

JVM :

- 1) it is used to provide resources like memory allocation.

interpreter :

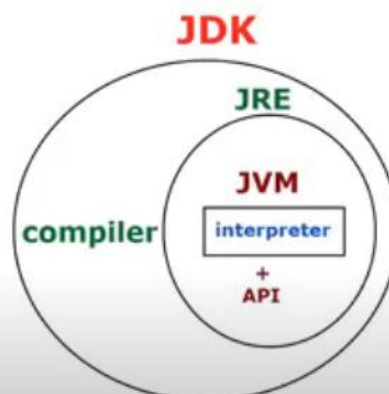
- 1) converts byte code into MC code.
- 2) it is used to execute byte code.

JDK , JRE , JVM

JDK => compiler + JRE

JRE => JVM + API

JVM => Interpreter



JDK => Java Development kit

JRE => Java RunTime Env.

JVM => Java Virtual MC.

API => Application Programming Interface

variables :

use :

-> To store data on computer memory variables are used.

define variables :

-> variable is a named memory location.

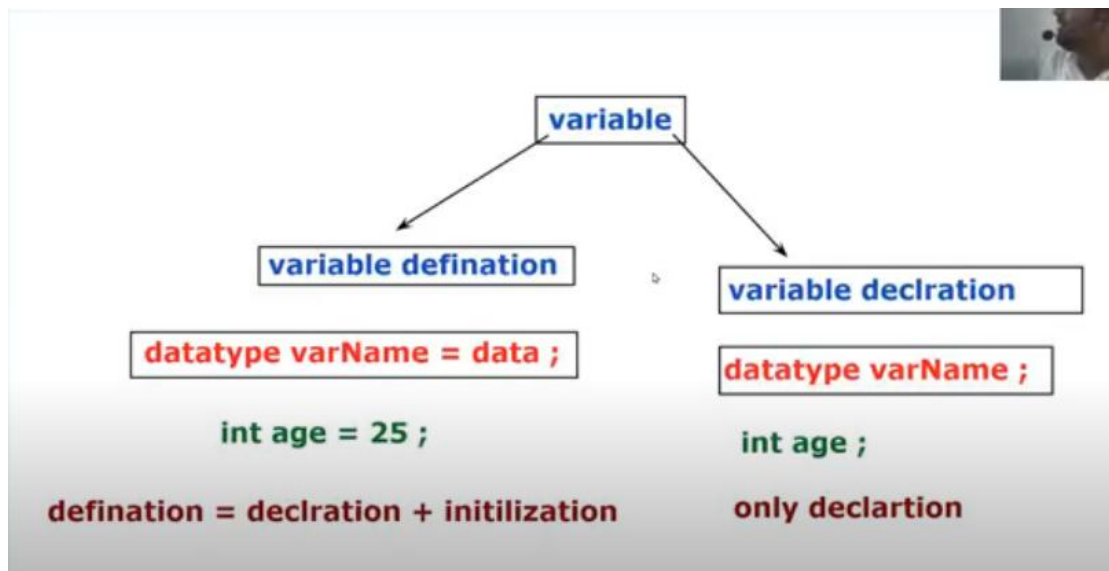
-> variables is like a container which is used to store data on computer memory.

Q- How to define variable in java ??

`datatype varname = value ;`

Q- How to declare variable in java ??

`datatype varname ;`



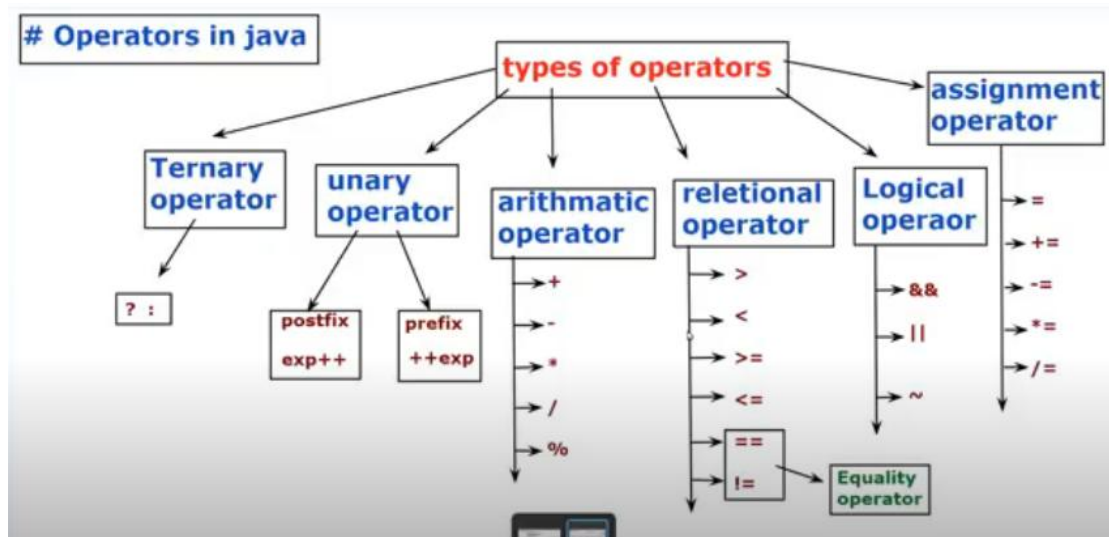
Reference Variable :

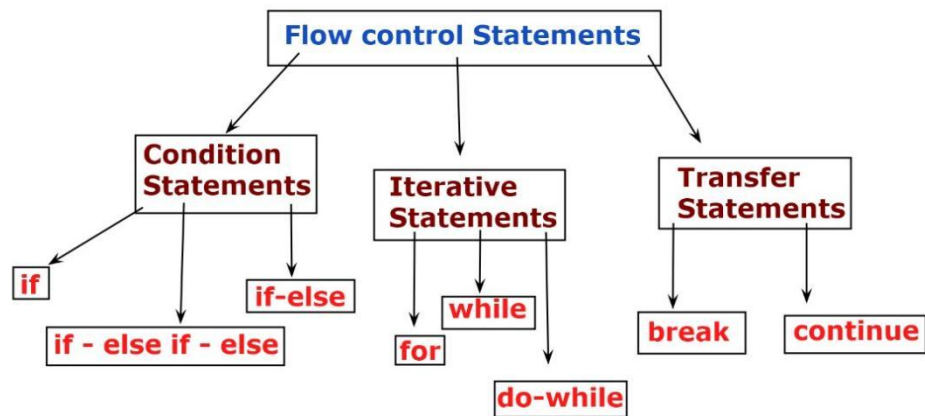
-> To access the functionality of an object.

-> To access the members of an objects
members -> [variables & methods]

Syntax :

`referenceVariable.members`





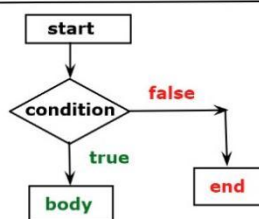
Conditional Statements :

-> according to the result of condition if we need to perform some kind action go for conditional statements.

if syntax :

```

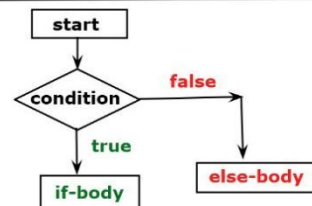
if (condition){
    // body
}
  
```



if-else syntax :

```

if (condition){
    // if-body
}else{
    // else-body
}
  
```

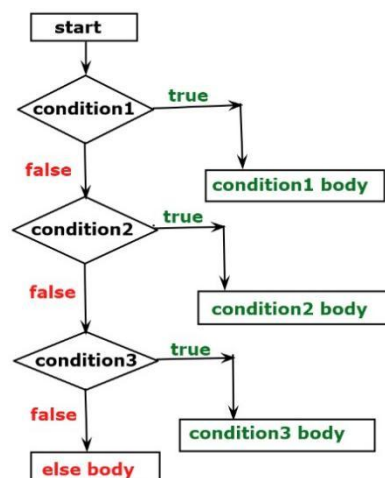


if - else if - else :

syntax:

```

if (condition1){
    // condition1 body
}
else if(condition2){
    // condition2 body
}
else if(condition3){
    // condition3 body
}
else{
    // else body
}
  
```



class :

- 1) class is a template or blueprint from which objects are created.
- 2) A class is a group of objects which have common properties and methods.
- 3) class determines how an object behaves and what an object will contain.
- 4) all objects in a given class will have same methods and properties.
- 5) class is used to define user-defined datatype.
- 6) class is a logical Entity.
- 7) class does not require memory for execution.
- 8) total information of an object is present in a class.
- 9) class is a referenced (non-primitive) datatype.

Syntax :

```
class ClassName {
    variables + methods + constructors + blocks
}
```

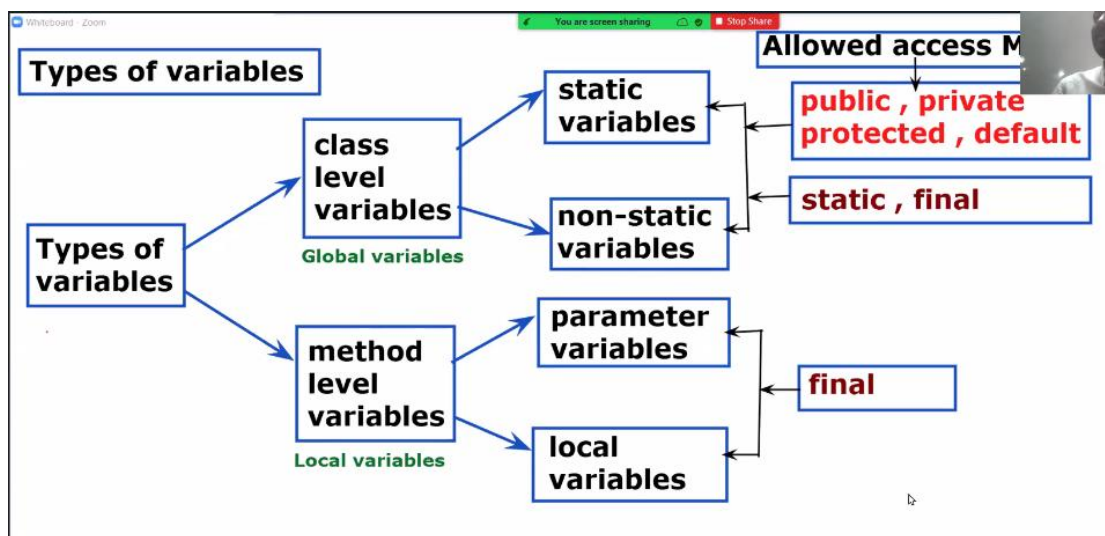
NOTE : ClassName must start with a capital letter only.

Object :

- 1) object is an instance of a class.
- 2) an object is a real-world Entity.
- 3) object is an entity which has state and behavior.
- 4) an object is a collection of data and methods.
- 5) an object is a member/product/instance of a class.
- 6) object is a physical Entity.
- 7) object requires memory for execution.

Syntax :

- 1] `ClassName referenceVariable = new ClassName();`
- 2] `ClassName referenceVariable;`
`referenceVariable = new ClassName();`
- 3] `var referenceVariable = new ClassName();`



Instance variable :

-> object level variables

-> if the value of variable is changing from object to object then that variable is called as instance variable.

-> for each object a separate copy of instance variable will be maintained at object level

Static variable :

-> class level variables

-> if the value of variable is same for all objects then that variable is called as static variable.

-> for all objects a single copy of static variables will be maintained at class level.

	static	non-static
place	inside class but outside of all methods , blocks and constructors	inside class but outside of all methods , blocks and constructors
definition	<code>static dataType varName ;</code>	<code>dataType varName ;</code>
creation	before object creation or .class file is loaded in JVM	after object creation
destroyed	.class file is removed from JVM	after object destroyed
access	using classname	using object reference
eg	<pre>class ClassName{ static collageName = "JBK" ; }</pre>	<pre>class ClassName{ String name ; double marks ; }</pre>

LOCAL VARIABLES :

```

returnType methodName(input parameters) {
    int a = 10 ;
    // body => variable declaration is local
}

```

The diagram shows a Java method signature and body. The **input parameters** in the signature are highlighted with a red box and labeled "local variables". The **int a = 10 ;** line in the body is also highlighted with a red box and labeled "local variable". A red arrow points from the "local variables" label to the input parameters, and another red arrow points from the "local variable" label to the **int a = 10 ;** line.

NOTE: Local variables are accessible within the method only. outside of method local variable we can not access.

Local Variables :

- 1) The variables which are declared inside a method body called as local variable.
- 2) Local variables are created when method execution is started.
- 3) Local variables are destroyed when method execution is finished.
- 4) There is no default values for local variables.
- 5) we can access local variables within the method body only. we can not access local variables outside of method body.

parameter variables

1) **sty** returnType methodName(**int x , int y**) {
}

2) **use:** To receive inputs we can use parameter variables.

3) when we can call method local variables are created.

4) once method execution is finished local variables are destroyed.

5) **scope :** within the method only.

local variables

```

returnType methodName( ){
    int x = 10;
    int y = 20;
}

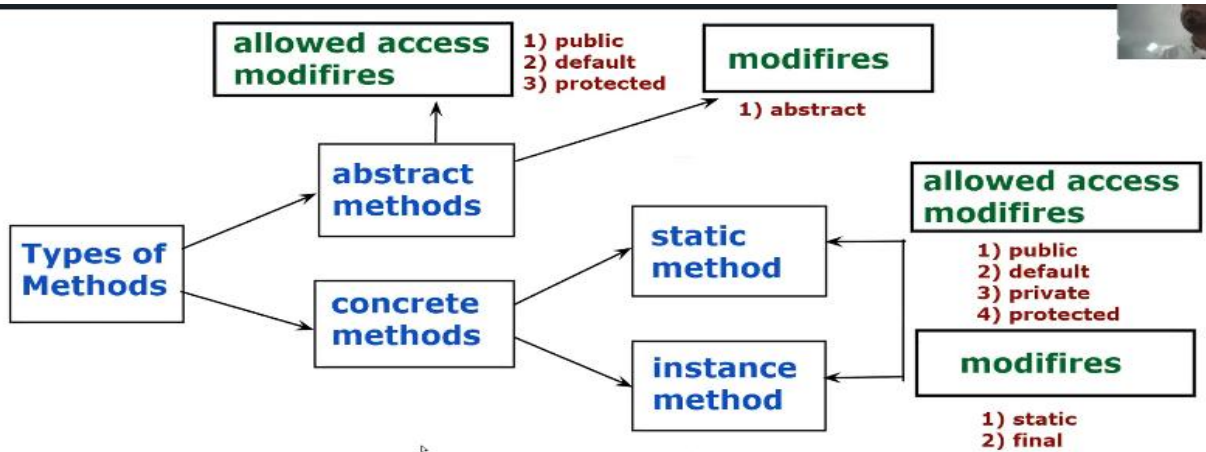
```

To perform business Logic or To perform some calculations

when we can call method local variables are created.

once method execution is finished local variables are destroyed.

scope : within the method only.



Methods

- 1) Method definition
- 2) Method Calling

sty for method definition :

```

returnType methodName(input args){
    -----
    processing of input args
    -----
}
  
```

sty for method calling :
method calling is depends on method type.

method definition(one) => method calling(many)

# Types of Methods		
-> use : To perform specific task.. -> methods are used to write bussiness Logic.		
	Types of Methods	
	static method	non-static method ✓
defination	<pre> static returnType methodName(input args){ // processing of static variables ✓ // processing of input args ✓ } </pre>	<pre> returnType methodName(input args){ // processing of static & non-static variables ✓ // processing of input args ✓ } </pre>
calling	ClassName.methodName() ;	objRef.methodName() ;

Constructor :

Purpose :

-> The main purpose of constructor is initialization of instance variable.

IMP Points :

- > constructor name and classname must be same.
- > constructor is a method only which does not have any return type including void.
- > for each object constructor will be executed only once.
- > constructor will be executed automatically when we create an object of class.
- > Two types of constructor :-> 1) default(no-arg) 2) parametrized (args)

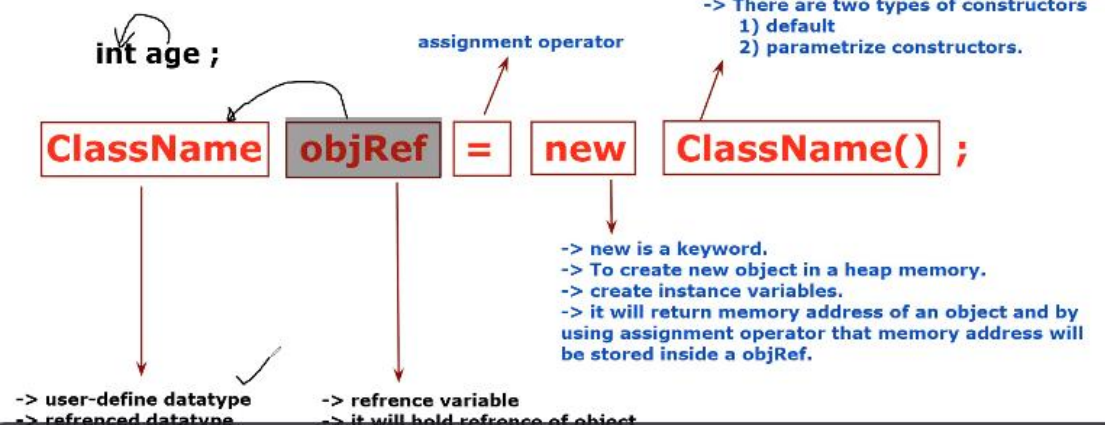
Syntax :

```
class ClassName{
    ClassName(input args){
        // initialization of instance variable.
    }
}
```

NOTES :

- 1) we can have public, private, protected and default constructor in java.
- 2) in java constructor can not be a final, abstract and static.
- 3) if there is no constructor in a class compiler automatically creates a default constructor.

Constructors :



difference between constructors and methods

	constructors	methods
1	constructors name and classname must be same.	method name can be anything.
2	constructors is not having any returntype including void.	method is having returntype.
3	for each object constructor will execute only once.	for each object method will execute any number of times.
4	constructors are used to initilized non-static variables.	To write bussiness logic we can use methods.
5	constructor will execute automatically when we create object.	we have to call methods to execute.
6	types :- A] Default , B] Parameterized	types :- A] static , B] non-static

this Keyword :

- > this is a keyword.
- > this is a reference variable that refers to current class object instance members.
- > we can use this keyword any number of times.

USE :

- > To access current class instance members.
- > instance members :-> instance variables and instance methods.

NOTE :

- > we can use this keyword anywhere except static method and static block.

this keyword :

- 1) this is a keyword.
- 2) this is a reference variable.
- 3) this keyword is used to refer current class instance members [instance variables and method].
- 4) this keyword we can use to differentiate between local and global variables.
- 5) we can use this keyword inside a instance method and inside a constructor only.

NOTE :

- 1) we can not use this keyword inside a static method or static block.

	Blocks	
	instance block	static block
definition	<pre>class ClassName{ { // instance block // access static + instance variables } }</pre>	<pre>class ClassName{ static{ // static block // we can access only static variable } }</pre>
execution	when we create object of class before constructors instance block will execute.	At the time of .class file loading in JVM memory static block will execute.

use of blocks : we can write time consuming code inside a blocks. like JDBC connectivity code.

Types of Inheritance :

- 1) single or simple
- 2) multi-level
- 3) Hierarchial
- 4) mutiple
- 5) Hybrid
- 6) Cyclic

NOTE :

1) java supports for single , multi-level and Hierarchial inheritance by using classes and interface.

2) java supports for mutiple, Hybrid inheritance by using interface.

single/simple Inheritance :

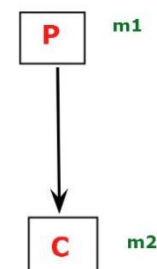
-> single parent & single child

```
class P{
    void m1(){
        sopln("parent class m1 method") ;
    }
}

class C extends P{
    void m2(){
        sopln("child class m2 method") ;
    }
}
```

```
P p1 = new P() ;
p1.m1() ;

C c1 = new C() ;
c1.m1() ;
c1.m2() ;
```



Inheritance :

-> The process where one class acquires the properties (variables and methods) of another class.

-> The process of acquire parent class properties in child class.

-> The process in wich one object can acquires all properties and methods of a parent class.

Syntax to implement inheritance in java :



Multi-level Inheritance :

-> single parent & single child but in multiple level.

```
class P{
    void m1(){
        sopln("parent class P -> m1 method") ;
    }
}

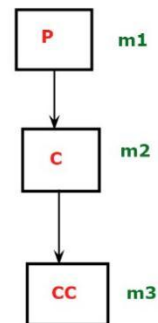
class C extends P{
    void m2(){
        sopln("child class C -> m2 method") ;
    }
}

class CC extends C{
    void m3(){
        sopln("child class CC -> m3 method") ;
    }
}
```

```
P p = new P() ;
p.m1() ;

C c = new C() ;
c.m1() ;
c.m2() ;

CC cc = new CC() ;
cc.m1() ;
cc.m2() ;
cc.m3() ;
```



hierarchical Inheritance :

-> single parent & multiple childs .

```
class P{
    void m1(){
        sopln("parent class p -> m1 method") ;
    }
}

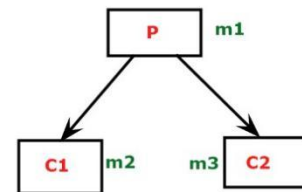
class C1 extends P{
    void m2(){
        sopln("child class c1 -> m2 method") ;
    }
}

class C2 extends P{
    void m3(){
        sopln("child class c2 -> m3 method") ;
    }
}
```

```
P p = new P() ;
p.m1() ;

C1 c1 = new C1() ;
c1.m2() ;
c1.m1() ;

C2 c2 = new C2() ;
c2.m3() ;
c2.m1() ;
```



super keyword

- 1) it is a non-static final reference variable.
- 2) super keyword holds super-class(parent) class information.
- 3) from sub-class if we want to refer super class instance members then use super keyword.
- 4) To differentiate subclass instance variables or instance methods from superclass instance variables and instance methods.

NOTE :

- 1) we can not use super keyword in static methods and static blocks.

this() & super() :

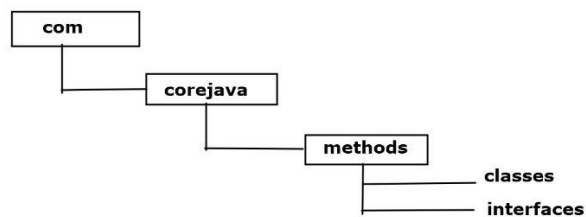
- 1) this() and super() both are constructor calls ,to call super class constructor and current class constructor.
- 2) this() and super() both we can use in constructor only as a first line only.
- 3) this() and super() we can use only one at a time.but not both simultaneously.

package and imports

- > package is a simple folder.
- > package is a encapsulation Mechanisum to group releted classes and interface into a single unit called as package.
- > in real time every class and interface is a part of package only.
- > in java prog we can write at most one package statement and package statement is first statement in your java programme.

package Name

package : com.corejava.methods



import statements :

p1

```

package p1 ;

class Test{

}
  
```

p2

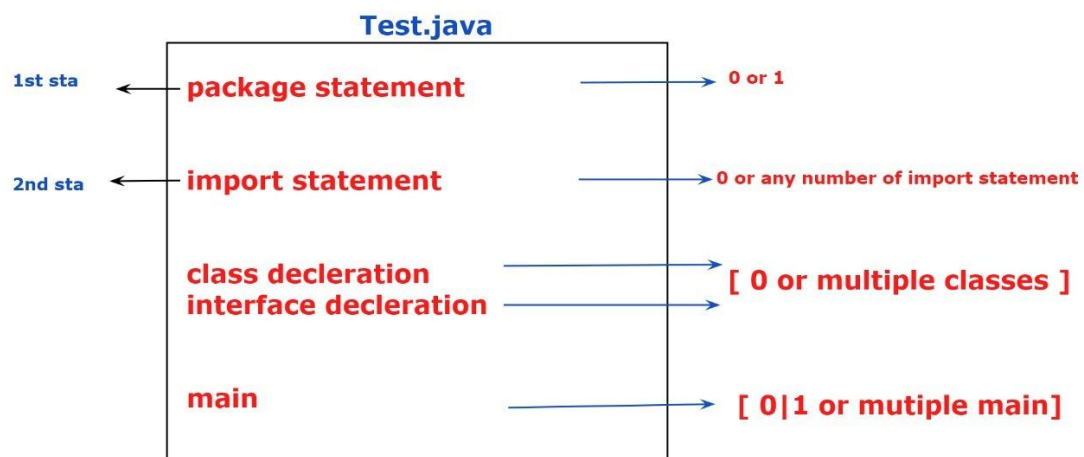
```

package p2 ;
import p1.Test

class Demo{

    p s v main(){
        var Test = new Test() ;
    }

}
  
```



Access Modifiers in Java :

- 1) private
- 2) <default> or <package-private>
- 3) protected
- 4) public

NOTE :

- 1) private , protected and public are the keywords.
- 2) default is not keyword it is just concept.

USE :

- > Access modifiers will tell us "where to access" class , variables and methods.
- > To set access permissions we can use Access Modifiers.

public (Global Level) :

Access : within the package + outside of the package.

Default (Package Level) :

package-private

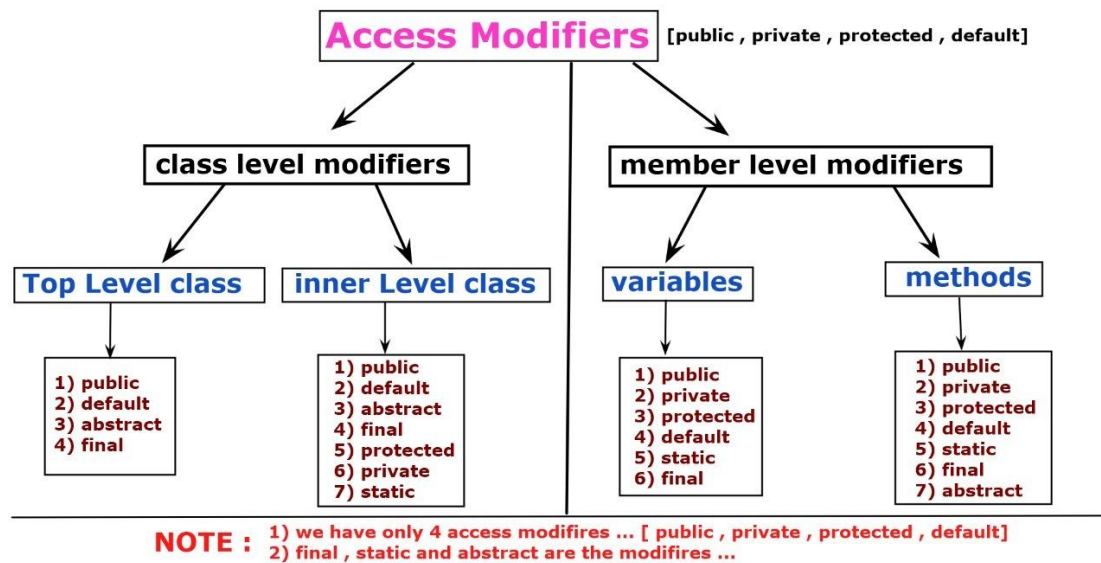
Access : within the package only.

private (Class Level) :

Access : within the class only.

protected :

Access : within the package + outside of package only in child classes using child reference.



```

class TopLevelClass{

}
  
```

```

class TopLevelClass{

    class InnerClass{

    }

}
  
```

sty :
AccessModifiers modifiers Name ;

```

1) public final int varName = value ;
2) public final void methodName(){

}
3) public final ClassName{

}
  
```

NOTE :

1) First check accessibility permissions of class.

2) secondly check accessibility permissions of members[variables and methods].